1. Background and Context

The NVIDIA Jetson Nano is a resource-constrained embedded platform intended for edge AI applications such as robotics, perception systems, and real-time vision pipelines. While PyTorch is commonly used during model development and training, deploying a raw .pt model directly on the Nano often leads to sub-optimal performance.

This is not a limitation of PyTorch as a framework, but rather a consequence of its design goals. PyTorch prioritizes flexibility and dynamic execution, which is ideal for research and experimentation but less suitable for deterministic, high-performance inference on embedded GPUs.

For deployment, NVIDIA provides TensorRT, an inference-specific optimization and runtime library designed to extract maximum performance from NVIDIA hardware. The following sections explain why TensorRT is the more appropriate choice for inference on Jetson Nano.

2. Constraints of Embedded Inference

Embedded systems differ fundamentally from desktop or server-class hardware. Compute capacity, memory bandwidth, and power budgets are tightly limited. As a result, inference frameworks must minimize overhead and make efficient use of available hardware.

Studies evaluating deep learning frameworks on Jetson platforms show that general-purpose runtimes such as PyTorch incur additional overhead during inference, particularly due to dynamic graph execution and less aggressive kernel optimization [1]. These overheads become noticeable on devices like the Jetson Nano, where margins are small and inefficiencies are amplified.

3. What TensorRT Changes in Practice

TensorRT is not a training framework. It is an inference optimizer and runtime. Its role is to take a trained model and transform it into a static execution engine tailored to the target GPU.

During engine generation, TensorRT performs:

- Layer and kernel fusion to reduce memory transfers
- Removal of redundant operations
- Precision lowering (FP16 or INT8 where supported)
- Hardware-specific kernel selection and scheduling

The result is a precompiled inference engine that executes with minimal runtime overhead. NVIDIA explicitly positions TensorRT as the preferred inference backend for Jetson devices, including Nano, within its optimized framework stack [2].

4. Performance Evidence on Jetson Platforms

Performance comparisons on Jetson-class hardware consistently show that TensorRT-based inference outperforms raw framework execution in terms of latency, throughput, and efficiency.

Benchmarking studies on NVIDIA Jetson platforms demonstrate that inference pipelines using TensorRT achieve lower inference latency and better hardware utilization compared to PyTorch running without TensorRT acceleration [1], [3]. These gains stem from TensorRT's static execution model and its ability to generate kernels optimized for the underlying GPU architecture.

Evaluations conducted on Jetson Xavier class devices further reinforce this trend, showing that TensorRT-accelerated inference frameworks deliver superior real-time performance compared to direct framework execution [4]. Although Xavier is more powerful than Nano, the relative benefit of TensorRT is often greater on smaller devices due to tighter resource constraints.

5. Precision Reduction and Efficiency

One of TensorRT's most practical advantages is its support for reduced-precision inference. By running models in FP16 or INT8 mode, TensorRT reduces memory bandwidth usage and computational load.

Research on quantized and reduced-precision inference shows that these techniques can significantly improve performance while maintaining acceptable accuracy when calibration is performed correctly [5]. On embedded platforms, this trade-off is often necessary to meet real-time constraints.

PyTorch supports reduced precision during training and inference, but it does not perform the same level of aggressive, hardware-specific optimization during runtime as TensorRT does.

6. Determinism and Deployment Stability

Another important consideration for embedded deployment is predictability. TensorRT engines are static: memory allocation, execution order, and kernel selection are fixed at build time. This leads to more consistent inference latency and reduced runtime variance.

In contrast, PyTorch's dynamic execution model introduces variability that may be acceptable in research environments but is undesirable in real-time systems such as mobile robots or control loops.

7. Practical Deployment on Jetson Nano

NVIDIA's official documentation explicitly recommends TensorRT as part of the deployment pipeline for deep learning applications on Jetson devices [2]. In practice, a common workflow is:

1. Train and validate the model in PyTorch
2. Export the model
3. Convert it to a TensorRT engine for deployment

This approach preserves development flexibility while ensuring efficient execution on the target hardware.

8. Summary

Using TensorRT engines instead of raw PyTorch .pt models for inference on the Jetson Nano is justified by both design intent and empirical evidence. TensorRT provides:

- Lower inference latency and higher throughput
- Reduced memory usage and improved efficiency
- Support for reduced-precision inference
- More deterministic execution behavior

For embedded inference workloads on Jetson Nano, TensorRT is not an optional optimization but a necessary step for achieving practical performance.

References

[1] D.–J. Shin and J.–J. Kim, “A Deep Learning Framework Performance Evaluation to Use YOLO in Nvidia Jetson Platform,” *Applied Sciences*, vol. 12, no. 8, Art. no. 3734, Apr. 2022.

[2] NVIDIA Corporation, “NVIDIA Optimized Frameworks,” NVIDIA Developer Documentation, 2025. Available: https://docs.nvidia.com/deeplearning/frameworks/index.html

[3] I. J. Ratul, Y. Zhou, and K. Yang, “Accelerating Deep Learning Inference: A Comparative Analysis of Modern Acceleration Frameworks,” *Electronics*, vol. 14, no. 15, Art. no. 2977, Jul. 2025.

[4] D.–J. Shin and J.–J. Kim, “Performance Evaluation of Deep Learning Frameworks on NVIDIA Jetson Platforms,” *Applied Sciences*, vol. 12, no. 8, 2022.

[5] A. Jain et al., “Efficient Execution of Quantized Deep Learning Models: A Compiler Approach,” *arXiv preprint arXiv:2006.10226*, 2020.